Лабораторна робота №5

Застосування нейро-нечітких мереж для прогнозування ризиків (керування виробничим процесом)

1. Мета роботи

Основною метою цієї роботи є глибоке ознайомлення з теоретичними основами та практичним застосуванням нечіткої логіки та гібридних нейро-нечітких систем (ANFIS). Навчитись виконувати ключові етапи роботи з нечіткими системами, як-от фазифікація та дефазифікація даних, а також реалізувати повноцінну модель для прогнозування ризиків у виробничому процесі, використовуючи сучасні бібліотеки Python, такі як **scikit-fuzzy** та **PyTorch**. Фінальним етапом є проведення експериментальних досліджень для оцінки ефективності розробленої моделі та формулювання обґрунтованих висновків.

Ключові завдання:

- **Сформувати нечіткі множини** для вхідних змінних (температура, тиск) та вихідної змінної «ризик».
- **Побудувати базу правил** типу IF-THEN, яка є ядром нечіткої системи.
- **Реалізувати ANFIS-модель** та порівняти її точність і гнучкість із класичною нечіткою системою.
- **Провести навчання та оцінку моделей**, візуалізувати результати за допомогою графіків (крива втрат, порівняння прогнозів) та розрахувати метрики ефективності (RMSE, MAE, R).

2. Теоретичний матеріал

2.1. Класична логіка проти нечіткої логіки

В основі цифрових обчислень лежить **класична (булева) логіка**, яка оперує виключно бінарними значеннями: **0** (хибність) або **1** (істина). Це ідеально підходить для опису однозначних станів. Наприклад, правило «Якщо температура перевищує 80°C, увімкнути охолодження» можна формалізувати так: стан «Висока температура» є істинним (1) при T > 80°C і хибним (0) в іншому випадку[9].

Однак реальний світ сповнений невизначеності та плавності. Людське мислення оперує такими поняттями, як «трохи тепло», «досить швидко» або «майже безпечно». **Нечітка логіка (Fuzzy Logic)**, запропонована Лотфі Заде, дозволяє математично описати таку невизначеність. Вона розширює класичну логіку, дозволяючи змінним приймати будь-яке значення в діапазоні **[0, 1]**. Це значення називається **ступенем належності** і показує, наскільки певний елемент належить до нечіткої множини.

- **Приклад:** За температури 75°C стан обладнання не є ні «середнім», ні «високим» в абсолютному сенсі. Нечітка логіка може описати це так: ступінь належності до множини «середня температура» дорівнює 0.4, а до множини «висока температура» — 0.6. Це дозволяє створювати більш гнучкі та людино-орієнтовані системи.

2.2. Основні етапи роботи нечіткої системи

Нечітка система висновку (Fuzzy Inference System, FIS) перетворює чіткі вхідні дані на чіткий вихід за допомогою нечітких правил. Цей процес складається з кількох ключових етапів.

1. Фазифікація (Fuzzification)

Це початковий етап, на якому відбувається перетворення чітких (числових) вхідних значень у нечіткі множини. Це досягається за допомогою **функцій належності (Membership Functions, MF)**, які кожному вхідному значенню ставлять у відповідність ступінь належності (від 0 до 1) до певної лінгвістичної категорії (наприклад, «низький», «середній», «високий»).

Найпоширеніші типи функцій належності:

- Трикутна **(trimf):** Проста функція, що задається трьома точками *a,m,b.*

$$f(x; a, m, b) = \max\left(\min\left(\frac{x-a}{m-a}, \frac{b-x}{b-m}\right), 0\right)$$

- Трапецієподібна **(trapmf):** Узагальнення трикутної, має плато і задається чотирма точками *a,b,c,d.*

$$f(x; a, b, c, d) = \max\left(\min\left(\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}\right), 0\right)$$

- Гаусова **(gaussmf):** Плавна, симетрична функція, що визначається центром c та шириною σ.

$$f(x; c, \sigma) = \exp\left(-\frac{(x-c)^2}{2\sigma^2}\right)$$

2. База правил та логічне виведення (Inference)

Ядром нечіткої системи є **база правил** — набір інструкцій у форматі **«ЯКЩО (умова), ТО (висновок)»**. Умова (антецедент) складається з однієї або кількох нечітких змінних, з'єднаних логічними операторами (AND, OR, NOT).

- **Приклад правила:** ЯКЩО температура 'висока' **І** тиск 'низький', ТО ризик 'високий'.

Для обчислення ступеня істинності умови використовуються t-норми (для AND) та s-норми (для OR). Найчастіше для **AND** використовується операція min, а для **OR** — max.

3. Агрегація (Aggregation)

На цьому етапі результати (висновки) всіх правил, що спрацювали, об'єднуються в єдину нечітку множину для кожної вихідної змінної. Це створює фінальну форму нечіткого виходу перед його перетворенням у чітке число.

4. Дефазифікація (Defuzzification)

Це фінальний етап, де агрегований нечіткий висновок перетворюється на єдине чітке числове значення, яке може бути використане для ухвалення рішень. Найпопулярніші методи:

- **Центр ваги (Centroid):** Найпоширеніший метод, який обчислює центр мас фінальної нечіткої множини. Він забезпечує плавний результат.

$$y^* = \frac{\int y \cdot \mu(y)\, dy}{\int \mu(y)\, dy}$$

- **Бісектор (Bisector):** Знаходить значення, яке ділить площу фігури на дві рівні частини.
- **Середнє з максимумів (Mean of Maximum, MOM):** Усереднює значення, при яких функція належності досягає свого максимуму.

Нейро-нечіткі мережі (ANFIS)

ANFIS (Adaptive Neuro-Fuzzy Inference System) — це гібридна інтелектуальна система, яка поєднує переваги нечіткої логіки та нейронних мереж. Вона використовує архітектуру нейронної мережі для автоматичного налаштування параметрів нечіткої системи на основі навчальних даних.

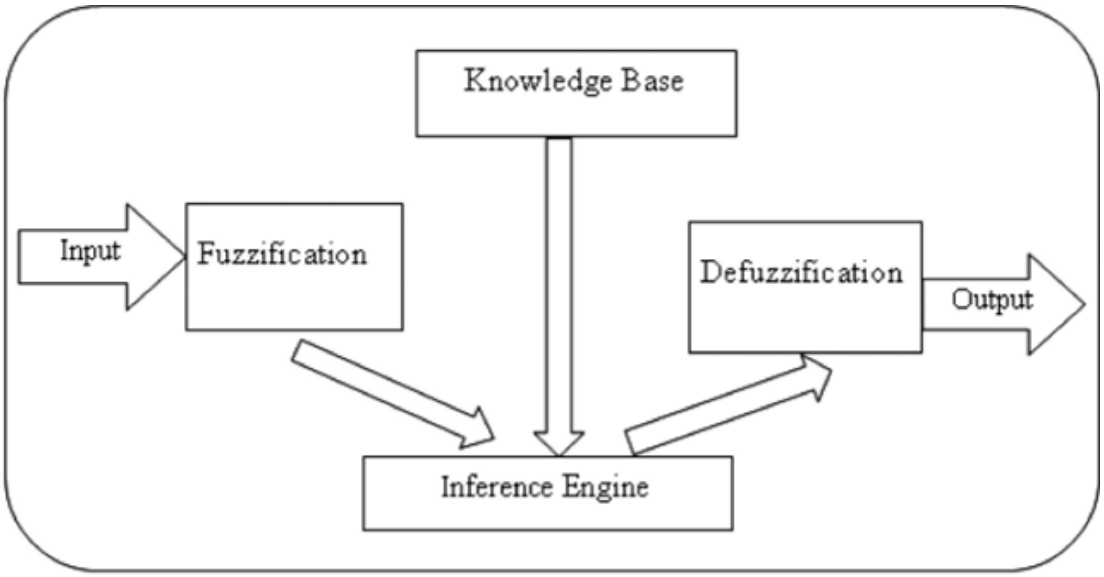
Структура та порівняння:

Характеристика	*Нечітка система*	*Нейронна мережа*	*ANFIS*
Інтерпретованість	Висока (правила зрозумілі людині)	Низька («чорний ящик»)	Висока (зберігає структуру правил) [28]
Гнучкість	Середня (правила задаються експертом)	Висока (адаптується до даних)	Висока (автоматичне налаштування) [29]
Навчання з даних	Обмежене або відсутнє	Так (основний принцип)	Так (гібридний алгоритм) [30]
Застосування	Системи керування, моделювання	Розпізнавання образів, прогнозування	Прогнозування та керування з інтерпретацією [31]

Архітектура ANFIS

ANFIS зазвичай реалізує нечітку систему типу Сугено (Sugeno), де висновок кожного правила є лінійною функцією від входів. Стандартна архітектура ANFIS складається з 5 шарів:

Структурна схема нейро-нечіткої системи (ANFIS):



- **Шар 1: Фазифікація.** Кожен вузол цього шару відповідає за обчислення ступеня належності вхідного сигналу до певної нечіткої множини (наприклад,). Параметри цього шару (центри та ширини функцій належності) називаються **параметрами передумови**.

- **Шар 2: Сила правила.** Вузли цього шару обчислюють «силу» або «вагу» кожного правила, перемножуючи ступені належності, отримані на першому шарі (операція t-норми).

$$w_r = \mu_{A_r}(x_1) \cdot \mu_{B_r}(x_2)$$

- **Шар 3: Нормалізація.** Вага кожного правила ділиться на суму ваг усіх правил, щоб отримати нормалізовані ваги. Це гарантує, що їхня сума дорівнює 1.

$$\bar{w}_r = \frac{w_r}{\sum_k w_k}$$

- **Шар 4: Локальний вихід (дефазифікація).** Кожен вузол обчислює висновок для одного правила, множачи нормалізовану вагу на лінійну комбінацію входів (модель Сугено).

$$f_r = \bar{w}_r \cdot (p_{r0} + p_{r1}x_1 + p_{r2}x_2)$$

Коефіцієнти називаються параметрами висновку.

- **Шар 5: Глобальний вихід.** Єдиний вузол, який підсумовує виходи всіх правил, формуючи кінцевий результат системи.

$$y = \sum_r f_r$$

Навчання ANFIS відбувається за допомогою **гібридного алгоритму**: на прямому проході за допомогою методу найменших квадратів налаштовуються параметри висновку (шар 4), а на зворотному — за допомогою градієнтного спуску коригуються параметри передумови (шар 1). Це дозволяє системі ефективно навчатися на даних.

2.3. Поглиблені теоретичні аспекти нейро-нечітких систем

Існуючий матеріал описує базові етапи роботи нечіткої системи (FIS) та 5-шарову архітектуру ANFIS. Для глибшого розуміння їх практичного застосування та обмежень, розглянемо ключові відмінності між основними типами FIS та більш детально проаналізуємо процес навчання.

2.3.1. Порівняння систем нечіткого виведення:

Мамдані та Сугено. У даній лабораторній роботі ми імплементуємо класичну систему типу Мамдані (Mamdani) за допомогою scikit-fuzzy та ANFIS, яка зазвичай базується на моделі типу Сугено (Sugeno, або Такагі-Сугено-Канг, TSK). Розуміння їх відмінностей є ключовим. Основна відмінність полягає у формі висновку (consequent) у базі правил. ***Система Мамдані:***

- Висновок (THEN-частина): є нечіткою множиною.
- Приклад правила: ЯКЩО температура 'висока' ТО ризик 'високий'.
- Тут «ризик 'високий'» — це така ж нечітка множина (наприклад, трикутна функція), як і вхідні.
- Процес: Як описано в розділі 2.2, система Мамдані вимагає «зрізання» (clipping) або «масштабування» (scaling) вихідної нечіткої множини відповідно до сили правила, потім агрегації (об'єднання) всіх «зрізаних» фігур і, нарешті, дефазифікації (наприклад, методом центру ваги) для отримання чіткого числа.
- Переваги: Дуже інтуїтивно зрозуміла, правила легко читаються та відповідають людській експертній логіці.
- Недоліки: Обчислювально складна. Розрахунок центру ваги (інтеграл) для складної агрегованої фігури є ресурсомістким.

Система Сугено (TSK):

- Висновок (THEN-частина): є математичною функцією (зазвичай лінійною або константою) від вхідних змінних.
- Тип 0 (TSK-0): Висновок є константою.
    - Приклад правила: ЯКЩО температура 'висока' І тиск 'низький' ТО ризик = 0.9.
- Тип 1 (TSK-1): висновок є лінійною функцією (саме цей тип реалізований в ANFIS у цій роботі).
    - Приклад правила: ЯКЩО температура 'висока' І тиск 'низький' ТО ризик = $0.7 \times T + 0.2 \times P + 0.1$.

Процес: Дефазифікація значно спрощена. Фінальний вихід системи — це зважена середня всіх виходів правил:

$$y = \frac{\sum_{r=1}^{N} w_r \cdot f_r}{\sum_{r=1}^{N} w_r} = \sum_{r=1}^{N} \overline{w_r} \cdot f_r$$

Де w_r — сила r-го правила (Шар 2), f_r — вихід функції r-го правила (наприклад, 0.9 або 0.7×T + 0.2×P + 0.1) 33, а w_r — нормалізована сила правила (Шар 3).

Переваги: обчислювально ефективна (немає складної дефазифікації), добре піддається оптимізації (навчанню), гарантує неперервність вихідної поверхні.

Недоліки: менш інтуїтивна; висновок у вигляді лінійної функції може бути складним для інтерпретації людиною-експертом.

Характеристика	Модель Мамдані	Модель Сугено (TSK)
Висновок правила (THEN)	Нечітка множина (напр., "Високий")	Функція від входів (напр., c або $ax+by+c$)
Агрегація	Об'єднання нечітких множин (max)	Не потрібна (тільки для дефазифікації)
Дефазифікація	Обчислювально складна (Центр ваги, Бісектор)	Проста (Зважена середня)
Інтерпретованість	Висока, "лінгвістична"	Середня, "математична"
Ефективність	Низька	Висока
Застосування	Експертні системи, прийняття рішень	Керування, прогнозування, ANFIS

У розділі 2.2. вказано, що ANFIS використовує гібридний алгоритм. Це найбільш суттєва перевага ANFIS, що поєднує швидкість методу найменших квадратів (LSE) та гнучкість градієнтного спуску. Розглянемо параметри, які ANFIS має налаштувати:

1. Параметри передумови (Premise Parameters): Це параметри функцій належності (Шар 1). Наприклад, для Гаусової функції це центр c та ширина σ. Вони визначають форму та положення нечітких множин.
2. Параметри висновку (Consequent Parameters): Це коефіцієнти лінійних функцій у моделі Сугено (Шар 4). Вони визначають вихід кожного правила.

Ці два типи параметрів мають різну природу. Вихід системи y (Шар 5) є лінійною функцією відносно параметрів висновку, але нелінійною функцією відносно параметрів передумови.Тому гібридний алгоритм розбиває кожну епоху навчання на два проходи.

- Прямий прохід (Forward Pass):
  - Вхідні дані проходять через мережу.
  - Параметри передумови (Шар 1) вважаються фіксованими.
  - Обчислюються нормалізовані ваги правил w_r (Шар 3).

Завданням стає знайти оптимальні параметри висновку $(p_{r0}, p_{r1}, \dots)$ для поточних функцій належності. Оскільки загальний вихід $y = \sum \overline{w_r} \cdot f_r = \sum \overline{w_r} \cdot (p_{r0} + p_{r1}x_1 + \cdots)$ є лінійною комбінацією параметрів p, ми отримуємо класичну задачу лінійної регресії.

Ця задача вирішується аналітично та швидко за один крок за допомогою Методу Найменших Квадратів (LSE). Ми миттєво знаходимо найкращі p для поточних c та σ.

- Зворотний прохід (Backward Pass):
    - Тепер параметри висновку (Шар 4), знайдені на кроці 1, вважаються фіксованими.
    - Обчислюється помилка (наприклад, MSE) між прогнозом y та реальним значенням.
    - Помилка поширюється назад по мережі.

Для оновлення параметрів передумови (Шар 1) використовується градієнтний спуск (або його варіації, як Adam, RMSprop), оскільки залежність помилки від них нелінійна. Градієнт показує, як потрібно змінити c та σ (змістити або розширити/звузити функції належності), щоб зменшити загальну помилку.

Отже, замість того, щоб шукати всі параметри лише повільним градієнтним спуском, ANFIS використовує LSE для миттєвого налаштування лінійної частини (висновків) і градієнтний спуск для повільного та точного налаштування нелінійної частини (функцій належності). Це робить навчання набагато швидшим та ефективнішим.

2.3.2. Практичні аспекти

Архітектура ANFIS є потужною, але на практиці стикається з двома серйозними викликами: "прокляттям розмірності" та перенавчанням.

Прокляття розмірності (Curse of Dimensionality) та генерація правил. ANFIS створює правила на основі всіх комбінацій нечітких множин для входів: 1) якщо у нас 2 входи (Температура, Тиск) і 3 функції належності для кожного (*'low', 'medium', 'high'*); 2) кількість правил $3 \times 3 = 9$ правил, це керована кількість; 3) якщо у нас 5 входів (додамо вологість, вібрацію, швидкість) і ті ж 3 ФН; 4) якщо у нас 10 входів і 3 ФН; 5) кількість правил = $3^5 = 243$ правила. Це називається «вибухом правил». Модель стає гігантською, вимагає величезної кількості пам'яті (для параметрів висновку p для кожного правила) і навчається надзвичайно довго або не навчається взагалі.

Рішення: Ініціалізація правил за допомогою кластеризації. Замість створення "сітки" правил (Grid Partitioning), можна видобути правила безпосередньо з даних. Найпопулярніший метод — нечітка кластеризація (Fuzzy C-Means, FCM).

- Кластеризація: Навчальні дані (пари вхід-вихід) подаються в алгоритм FCM.
- Пошук центрів: Алгоритм знаходить *K* (задана кількість правил) центрів кластерів у просторі даних. Кожен центр кластера — це "прототип" певної поведінки системи.
- Генерація правил: Кожен центр кластера перетворюється на одне нечітке правило.
    - Параметри передумови (Шар 1): Функції належності (наприклад, Гаусові) ініціалізуються так, щоб їхні центри c збігалися з координатами центру кластера, а ширина σ— з «розкидом» кластера.
    - Параметри висновку (Шар 4): Можуть бути ініціалізовані нулями або середнім значенням виходу в кластері.

Таким чином, ми отримуємо компактну систему з *K* правил (наприклад, 10-20), яка відображає реальну структуру даних, а не сліпо покриває весь простір.

Боротьба з перенавчанням (Overfitting)

Як і будь-яка нейронна мережа, ANFIS може «запам'ятати» навчальні дані, але погано узагальнювати їх на нових, невідомих даних (тестовій вибірці).

Рання зупинка (Early Stopping): це найпоширеніший метод. Наприклад, відстежуємо помилку на валідаційній (тестовій) вибірці (*val_loss*) під час навчання. Навчання зупиняється, коли помилка на валідації починає зростати, навіть якщо помилка на навчальній вибірці (*train_loss*) продовжує падати. Це означає, що модель почала перенавчатися.

Регуляризація (Regularization): до функції втрат додається "штраф" за надто великі значення параметрів (ваг). Це змушує модель шукати "простіші" рішення, роблячи функції належності більш гладкими, а параметри висновку — меншими. Найпоширеніший тип — L2-регуляризація (*weight_decay* в оптимізаторах PyTorch). Перевірка кількості правил: Занадто велика кількість правил (або занадто гнучкі функції належності) призводить до перенавчання. Потрібно знайти баланс: достатньо правил, щоб описати складність процесу, але не надто багато, щоб не «вивчити шум».

3. Практична частина

3.1. Постановка задачі та генерація даних

Для моделювання ми створимо синтетичний набір даних, що імітує залежність рівня ризику на виробництві від трьох ключових факторів:

1. **Температура (T):** від 20 до 100 °C.
2. **Тиск (P):** від 1 до 10 бар.
3. **Вологість (H):** від 10 до 90 %.

Вихідним значенням є **рівень ризику (Risk)** відмови обладнання — число в діапазоні від 0 (немає ризику) до 1 (максимальний ризик).

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import torch
import torch.nn as nn
import skfuzzy as fuzz
from skfuzzy import control as ctrl
import matplotlib.pyplot as plt

# Генерація даних
np.random.seed(42)
n_samples = 1200
temp = np.random.uniform(20, 100, n_samples)  # Температура, °C
press = np.random.uniform(1, 10, n_samples)    # Тиск, Бар
```

```
humid = np.random.uniform(10, 90, n_samples)  # Вологість, %

# Цільова змінна ризику (складна залежність з шумом)
risk = (0.5 * (temp / 100) +
        0.3 * (1 - press / 10) +
        0.2 * (humid / 100) +
        0.1 * np.sin(temp / 10) +
        np.random.normal(0, 0.03, n_samples))
risk = np.clip(risk, 0, 1)

# Нормалізація даних до діапазону [0, 1]
X = np.vstack([temp, press, humid]).T
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X)

# Розбиття на навчальну та тестову вибірки
X_train, X_test, y_train, y_test = train_test_split(
    X_normalized, risk, test_size=0.2, random_state=42
)

print(f"Розмір навчальної вибірки: {X_train.shape[0]}")
print(f"Розмір тестової вибірки: {X_test.shape[0]}")
```

3.2. Реалізація класичної нечіткої системи (Mamdani)

Спочатку побудуємо просту систему нечіткого виведення типу Мамдані за допомогою scikit-fuzzy.

```
# Визначення універсумів та нечітких змінних
t_norm = ctrl.Antecedent(np.arange(0, 1.01, 0.01), 'temperature')
p_norm = ctrl.Antecedent(np.arange(0, 1.01, 0.01), 'pressure')
risk_out = ctrl.Consequent(np.arange(0, 1.01, 0.01), 'risk')

# Визначення функцій належності (трикутні)
t_norm['low'] = fuzz.trimf(t_norm.universe, [0, 0, 0.4])
t_norm['medium'] = fuzz.trimf(t_norm.universe, [0.2, 0.5, 0.8])
t_norm['high'] = fuzz.trimf(t_norm.universe, [0.6, 1.0, 1.0])

p_norm['low'] = fuzz.trimf(p_norm.universe, [0, 0, 0.4])
p_norm['medium'] = fuzz.trimf(p_norm.universe, [0.2, 0.5, 0.8])
p_norm['high'] = fuzz.trimf(p_norm.universe, [0.6, 1.0, 1.0])

risk_out['low'] = fuzz.trimf(risk_out.universe, [0, 0, 0.5]) [cite: 207]
risk_out['medium'] = fuzz.trimf(risk_out.universe, [0.2, 0.5, 0.8])
risk_out['high'] = fuzz.trimf(risk_out.universe, [0.5, 1, 1])

# Візуалізація функцій належності
t_norm.view()
p_norm.view()
risk_out.view()
```

```
# Формулювання бази правил
rule1 = ctrl.Rule(t_norm['high'] & p_norm['low'], risk_out['high'])
rule2 = ctrl.Rule(t_norm['medium'] & p_norm['medium'], risk_out['medium'])
rule3 = ctrl.Rule(t_norm['low'] & p_norm['high'], risk_out['low'])

# Створення системи керування
risk_ctrl_system = ctrl.ControlSystem([rule1, rule2, rule3])
risk_simulator = ctrl.ControlSystemSimulation(risk_ctrl_system)

# Приклад розрахунку для одного значення
sample_idx = 10
risk_simulator.input['temperature'] = X_test[sample_idx, 0]
risk_simulator.input['pressure'] = X_test[sample_idx, 1]
risk_simulator.compute()

print(f"Прогнозований ризик (Fuzzy): {risk_simulator.output['risk']:.4f}")
print(f"Реальний ризик: {y_test[sample_idx]:.4f}")
risk_out.view(sim=risk_simulator)
```

3.3. Реалізація та навчання ANFIS моделі (PyTorch)

Тепер реалізуємо ANFIS-подібну модель, яка зможе навчатися на даних. Це буде нейро-нечіткий перцептрон, де перший шар імітує фазифікацію, а другий — агрегацію правил та дефазифікацію за типом Сугено.

```
# Перетворення даних у тензори PyTorch
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

# Архітектура моделі ANFIS
class SimpleANFIS(nn.Module):
    def __init__(self, n_inputs=3, n_rules=8):
        super().__init__()
        # Шар, що імітує центри функцій належності
        self.premise = nn.Linear(n_inputs, n_rules, bias=True)
        # Параметр, що відповідає за ширину функцій (навчається)
        self.scale = nn.Parameter(torch.rand(n_rules))
        # Лінійний шар для комбінації виходів правил (Sugeno)
        self.mix = nn.Linear(n_rules, 1, bias=True)

    def forward(self, x):
        # 1. Фазифікація (гаусоподібні активації)
        a = self.premise(x)
        activations = torch.exp(-0.5 * (a / (1e-6 + self.scale))**2)

        # 2. Нормалізація ваг правил
        weights = activations / (1e-6 + activations.sum(dim=1, keepdim=True))
```

```
        # 3. Дефазифікація (вихід моделі Сугено)
        y = self.mix(weights)

        # Обмеження виходу в діапазоні [0, 1]
        return torch.sigmoid(y)

# Налаштування та тренування моделі
model_anfis = SimpleANFIS(n_inputs=3, n_rules=10)
optimizer = torch.optim.Adam(model_anfis.parameters(), lr=0.01)
loss_fn = nn.MSELoss()

epochs = 1000
train_losses, val_losses = [], []

for epoch in range(epochs):
    model_anfis.train()
    y_pred = model_anfis(X_train_t)
    loss = loss_fn(y_pred, y_train_t)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 100 == 0:
        model_anfis.eval()
        with torch.no_grad():
            y_val_pred = model_anfis(X_test_t)
            val_loss = loss_fn(y_val_pred, y_test_t)
        train_losses.append(loss.item())
        val_losses.append(val_loss.item())
        print(f'Епоха {epoch+1}/{epochs}, Train Loss: {loss.item():.5f}, Val Loss: {val_loss.item():.5f}')
```

3.4. Оцінка та візуалізація результатів

Після навчання моделі оцінимо її ефективність за допомогою стандартних метрик регресії та візуалізуємо її прогнози.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

model_anfis.eval()
with torch.no_grad():
    y_pred_final = model_anfis(X_test_t).numpy().flatten()

y_true_final = y_test

# Розрахунок метрик
rmse = mean_squared_error(y_true_final, y_pred_final, squared=False)
```

```
mae = mean_absolute_error(y_true_final, y_pred_final)
r2 = r2_score(y_true_final, y_pred_final)

print("\nМетрики якості моделі ANFIS:")
print(f"  RMSE: {rmse:.4f}")
print(f"  MAE: {mae:.4f}")
print(f"  R²: {r2:.4f}")

# Візуалізація результатів
plt.figure(figsize=(14, 6))

# Графік 1: Порівняння істинних та передбачених значень
plt.subplot(1, 2, 1)
plt.plot(y_true_final[:100], 'o-', label='Істинні значення', alpha=0.7)
plt.plot(y_pred_final[:100], 'o-', label='Прогнози ANFIS', alpha=0.7)
plt.title('Порівняння прогнозів та реальних даних')
plt.xlabel('Зразки з тестової вибірки')
plt.ylabel('Рівень ризику')
plt.legend()
plt.grid(True)

# Графік 2: Діаграма розсіяння
plt.subplot(1, 2, 2)
plt.scatter(y_true_final, y_pred_final, alpha=0.5, s=15)
plt.plot([0, 1], [0, 1], 'r--', label='Ідеальний прогноз')
plt.title('Діаграма розсіяння')
plt.xlabel('Істинні значення')
plt.ylabel('Прогнозовані значення')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

4. Індивідуальні завдання

Оберіть завдання відповідно до вашого номера у списку групи. Виконайте його, детально опишіть хід роботи, додайте необхідні графіки та таблиці, а також зробіть висновок за результатами.

№	Batch size	Epochs	Оптимізатор	Додаткове завдання
1	32	100	Adam	**Порівняти різні функції належності (трикутна, гаусова).**
2	64	200	SGD	**Побудувати confusion matrix.**

3	128	150	RMSprop	**Візуалізувати приклади прогнозів.**
4	256	300	Adam	**Порівняти нечітку систему та ANFIS.**
5	40	100	SGD	**Вивести приклади правильно класифікованих значень.**
6	54	200	Adam	**Дослідити вплив кількості правил.**
7	75	150	RMSprop	**Використати heatmap важливості змінних.**
8	100	200	Adam	**Додати новий фактор (вологість).**
9	32	250	RMSprop	**Візуалізувати помилкові передбачення.**
10	64	300	Adam	**Дослідити вплив різних методів дефазифікації.**
11	128	200	SGD	**Побудувати графік зміни похибки (loss curve).**
12	256	100	Adam	**Порівняти точність при різних batch size.**
13	40	150	RMSprop	**Зробити аналіз впливу кількості епох.**
14	54	250	Adam	**Побудувати ROC-криву та обчислити AUC.**
15	75	300	SGD	**Додати новий вихід (категорія ризику).**
16	100	150	RMSprop	**Виконати порівняння ANFIS із класичною нейронною мережею (MLP).**
17	32	200	Adam	**Дослідити вплив L2-регуляризації.** Додайте weight_decay в оптимізатор (напр.,

				1e-4). Порівняйте метрики та криві навчання з моделлю без регуляризації.
18	64	250	RMSprop	**Порівняти оптимізатори Adam та SGD.** Навчіть дві однакові ANFIS-моделі з різними оптимізаторами. Порівняйте швидкість збіжності та фінальну якість.
19	128	100	Adam	**Проаналізувати вивчені функції належності.** Спробуйте візуалізувати, як виглядають "центри" правил (ваги self.premise.weight). Побудуйте 3D scatter plot, де кожна точка - це правило.
20	256	200	SGD	**Застосувати модель до даних з викидами.** Згенеруйте новий набір даних, додавши аномальні значення до кількох зразків. Навчіть модель і проаналізуйте, як викиди вплинули на її точність.
21	40	300	Adam	**Реалізувати ANFIS типу TSK-1.** Модифікуйте шар self.mix, щоб він приймав не тільки ваги правил, а й вхідні дані x, реалізуючи повну лінійну модель Сугено першого порядку.
22	54	150	RMSprop	**Порівняти ANFIS з методом k-найближчих сусідів (k-NN).** Навчіть регресор KNeighborsRegressor з scikit-learn і порівняйте його метрики з ANFIS.
23	75	250	Adam	**Дослідити вплив шуму в даних.** Згенеруйте дані з різним рівнем шуму (збільште стандартне відхилення в np.random.normal). Побудуйте графік залежності RMSE від рівня шуму.
24	100	100	SGD	**Використати іншу функцію втрат (MAE).** Замініть nn.MSELoss на nn.L1Loss (Mean Absolute Error). Порівняйте результати та швидкість навчання.

25	32	300	Adam	**Порівняти ANFIS з градієнтним бустингом.** Навчіть модель GradientBoostingRegressor з scikit-learn. Порівняйте її точність, швидкість та інтерпретованість з ANFIS.

Рекомендації щодо налаштувань

- Починайте з 5–8 правил; збільшуйте лише за потреби.
- Тип ФН: gauss для гладких меж, trimf для простоти/інтерпретації.
- Оптимізатор: Adam (швидка збіжність), RMSprop (стабільний на «шумних» даних), SGD (для контрольованих експериментів).
- Регуляризація: weight_decay, рання зупинка за val-втратою.
- Для класифікації ризику – встановіть поріг 0.5 або підберіть його за максимальним F1/Youden J з ROC.

5. Зміст звіту

1. **Титульний аркуш.**
2. **Постановка задачі:** Опис мети, вхідних та вихідних даних, методології.
3. **Теоретичні відомості:** Стислий опис принципів нечіткої логіки, архітектури ANFIS, функцій належності та методів дефазифікації.
4. **Програмна реалізація:** Ключові фрагменти коду для генерації даних, реалізації нечіткої системи та ANFIS-моделі з коментарями.
5. **Результати експериментів:**
   - Виконання індивідуального завдання.
   - Графіки (функції належності, криві навчання, порівняння прогнозів тощо).
   - Таблиці з фінальними метриками якості моделей.
6. **Висновки:** Аналіз отриманих результатів. Оцінка ефективності та придатності нейро-нечітких систем для задач прогнозування ризиків. Порівняння з іншими підходами.